

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.13

RELATION QUERIES (JOINS)

 Copy as Markdown

Relation queries (Joins)

The Serverpod query framework supports filtering on, sorting on, and including relational data structures. In SQL this is often achieved using a join operation. The functionality is available if there exists any [one-to-one](#) or [one-to-many](#) object relations between two models.

Include relational data

To include relational data in a query, use the `include` method. The `include` method has a typed interface and contains all the declared relations in your yaml file.

```
var employee = await Employee.db.findById(  
  session,  
  employeeId,  
  include: Employee.include(  
    address: Address.include(),  
  ),  
);
```

The example above returns an employee including the related address object.

Nested includes

It is also possible to include deeply nested objects.

```
var employee = await Employee.db.findById(  
  session,  
  employeeId,  
  include: Employee.include(  
    company: Company.include(  
      address: Address.include(),  
    ),  
  ),  
);
```



```
);
```

The example above returns an employee including the related company object that has the related address object included.

Any relational object can be included or not when making a query but only the includes that are explicitly defined will be included in the result.

```
var employee = await Employee.db.findById(
  session,
  employeeId,
  include: Employee.include(
    address: Address.include(),
    company: Company.include(
      address: Address.include(),
    ),
  ),
);
```

The example above includes several different objects configured by specifying the named parameters.

Include relational lists

Including a list of objects (1:n relation) can be done with the special `includeList` method. In the simplest case, the entire list is included.

```
var company = await Company.db.findById(
  session,
  companyId,
  include: Company.include(
    employees: Employee.includeList(),
  ),
);
```

The example above returns a company with all related employees included.

Nested includes

The `includeList` method takes its nested includes through an `include` parameter, the same name used on `findById`. When including something on a list it means that every entry in the list will each have access to the nested object.

```
var company = await Company.db.findById(
  session,
  companyId,
  include: Company.include(
```

```

    employees: Employee.includeList(
      include: Employee.include(
        address: Address.include(),
      ),
    ),
  ),
);

```

The example above returns a company with all related employees included. Each employee will have the related address object included.

It is even possible to include lists within lists.

```

var company = await Company.db.findById(
  session,
  companyId,
  include: Company.include(
    employees: Employee.includeList(
      include: Employee.include(
        tools: Tool.includeList(),
      ),
    ),
  ),
);

```

The example above returns a company with all related employees included. Each employee will have the related tools list included.

NOTE

For each call to `includeList` (nested or not) the Serverpod Framework will perform one additional query to the database.

Filter and sort

When working with large datasets, it's often necessary to [filter](#) and [sort](#) the records to retrieve the most relevant data. Serverpod offers methods to refine the included list of related objects:

Filter

Use the `where` clause to filter the results based on certain conditions.

```

var company = await Company.db.findById(
  session,
  companyId,
  include: Company.include(
    employees: Employee.includeList(

```

```
    where: (t) => t.name.ilike('a%')
  ),
),
);
```

The example above retrieves only employees whose names start with the letter 'a'.

Sort

The orderBy clause lets you sort the results based on a specific field.

```
var company = await Company.db.findById(
  session,
  companyId,
  include: Company.include(
    employees: Employee.includeList(
      orderBy: (t) => t.name,
    ),
  ),
);
```

The example above sorts the employees by their names in ascending order.

Pagination

[Paginate](#) results by specifying a limit on the number of records and an offset.

```
var company = await Company.db.findById(
  session,
  companyId,
  include: Company.include(
    employees: Employee.includeList(
      limit: 10,
      offset: 10,
    ),
  ),
);
```

The example above retrieves the next 10 employees starting from the 11th record:

Combined, these methods let you query, filter, and sort relational data efficiently.

Attach and detach relations

Managing relationships between tables is a common task. Serverpod provides methods to link (attach) and unlink (detach) related records:

Attach single row

Link an individual employee to a company. This operation associates an employee with a specific company:

```
var company = await Company.db.findById(session, companyId);
var employee = await Employee.db.findById(session, employeeId);

await Company.db.attachRow.employees(session, company!, employee!);
```

Bulk attach rows

For scenarios where you need to associate multiple employees with a company at once, use the bulk attach method. This operation is atomic, ensuring all or none of the records are linked:

```
var company = await Company.db.findById(session, companyId);
var employee = await Employee.db.findById(session, employeeId);

await Company.db.attach.employees(session, company!, [employee!]);
```

Detach single row

To remove the association between an employee and a company, use the detach row method:

```
var employee = await Employee.db.findById(session, employeeId);

await Company.db.detachRow.employees(session, employee!);
```

Bulk detach rows

In cases where you need to remove associations for multiple employees simultaneously, use the bulk detach method. This operation is atomic:

```
var employee = await Employee.db.findById(session, employeeId);

await Company.db.detach.employees(session, [employee!]);
```

NOTE

When using the attach and detach methods the objects passed to them have to have the `id` field set.

The detach method is also required to have the related nested object set if you make the call from the side that does not hold the foreign key.